

Unit 1

Basic Syntactical Constructs In Java

➤ What is Java

Java is a high level, robust, object-oriented and secure programming language.

Java was developed by Sun Microsystems (which is now the subsidiary of Oracle) in the year 1995.

James Gosling is known as the father of Java. Before Java, its name was **Oak**.

Since Oak was already a registered company, so James Gosling and his team changed the Oak name to Java.



Sample Java Program

```
class Sample
{
    public static void main (String args[])
    {
        System.out.println("Hello");
    }
}
```

A] Class Declaration

The first line

```
class Sample
```

declares a class, which is an object-oriented construct.

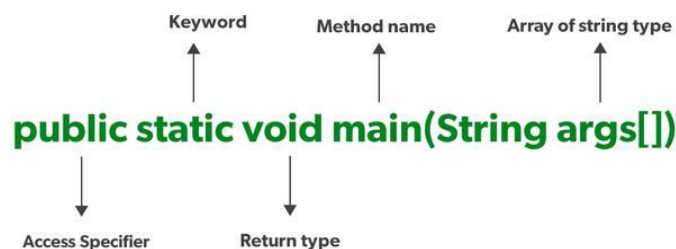
Java is a true object-oriented language **and** therefore, **everything** must be placed inside a class. **class** is a **keyword** and declares **that a new class** definition follows.

Sample is a Java identifier that specifies the name **of** the class **to be defined**.

B] Opening Brace

Every class definition in Java begins with an opening brace "{" and ends with a matching closing brace "}". This is similar to C++ class construct (Note that class definition in C++ ends with a semicolon.)

C] The Main Line



Every **Java** application program must include the `main()` method.

This is the starting point for the interpreter to begin the execution of the program.

A Java application can have any number of classes but only one of them must include a **main** method to initiate the execution.

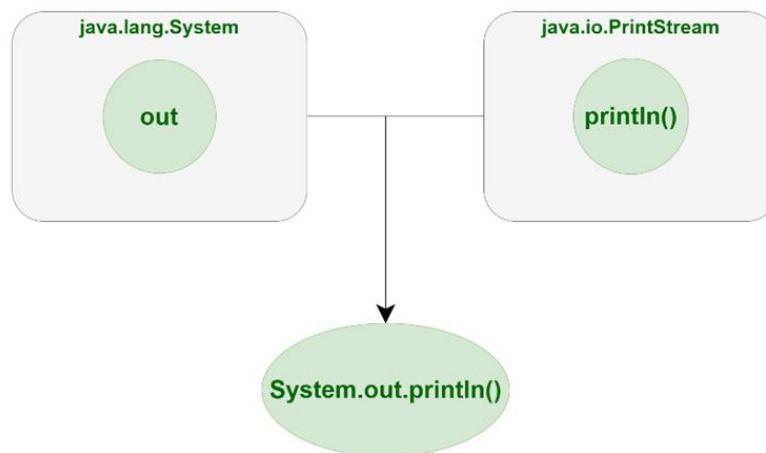
This line contains number of keywords, **public**, **static** and **void**.

public:	It is an Access modifier, which specifies from where and who can access the method. main() method is made public so that JVM can invoke it from outside the class. If we use private, protected, and default before the main() method, it will not be visible to JVM.
static:	The main() method is marked static so that the JVM may call it without having to create an object of the class that contains the main() method.
void:	It is a returntype and is used to specify that main() method doesn't return anything.

Here, **String args[]** declares a parameter named **args**, which contains an array of objects of the class type **String**.

DJ The Output Line

System.out.println() is used to print an argument that is passed to it. The statement can be broken into 3 parts which can be understood separately as:



System:

It is a built in class defined in the java.lang package.

It provides access to standard input, output

out:

This is an object of `PrintStream` type, which is a public and static member field of the `System` class.

It handles the actual writing of data to the output destination.

As it is static, it can be accessed directly using the class name (`System.out`).

println():

It prints any argument passed to it and adds a new line to the output.

This is an upgraded version of `print()`.

Every Java statement must end with a semicolon.

➤ Implementing A JAVA Program

Implementation of a Java application program involves a series of steps. They include:

- 1 Creating the program**
- 2 Compiling the program**
- 3 Running the program**

Remember that, before we begin creating the program, the Java Development Kit (JDK) must be properly installed on our system.

1. Creating the Program

We can create a program using any text editor. Assume that we have entered the following program:

```
class Test
{
    public static void main (String args[])
    {
        System.out.println("Hello!");
        System.out.println("Welcome to the world of Java.");
        System.out.println("Let us learn Java.");
    }
}
```

We must save this program in a file called **Test.java** ensuring that the filename contains the class name properly. This file is called the source file. Note that all Java source files will have the extension java. Note also that if a program contains multiple classes, the file name must be the classname of the class containing the main method.

2. Compiling the Program

To compile the program, we must run the Java Compiler **javac**, with the name of the source file on the command line as shown below:

```
javac Test.java
```

If everything is OK, the **javac** compiler creates a file called **Test.class** containing the bytecodes of the program. Note that the compiler automatically names the bytecode file as

```
<classname> .class
```

3. Running the Program

We need to use the Java interpreter to run a standalone program. At the command prompt, type

```
java Test
```

Now, the interpreter looks for the main method in the program and begins execution from there. When executed, our program displays the following:

```
Hello!
Welcome to the world of Java.
Let us learn Java.
```

Note that we simply type "**Test**" at the command line and not "Test.class" or "Test.java".

➤ Features of Java

Q] What are the features of JAVA?

[W-08, 09, 10, 11, 12, S-12]

Q] Explain: 1) Platform independence

2) Compiled and interpreted features of Java.

[W-14]

1. Compiled and Interpreted

Usually a computer language is either compiled or interpreted. Java combines both these approaches thus making Java a two-stage system. First, Java compiler translates **source code** into **byte code** instructions. **Byte codes** are **not machine instructions** and therefore, in the second stage, **Java interpreter** generates **machine code** that can be **directly executed by the machine** that is running the Java program.

We can thus say that Java is **both a compiled and an interpreted language**.

2. Platform-Independent and Portable

Q] Explain the concept of platform independence and portability with respect to Java language.

(S-19) [4 Marks]

Java is a **platform independent** language. This is possible because when a **java program** is **compiled**, an **intermediate code** called the **byte code** is obtained.

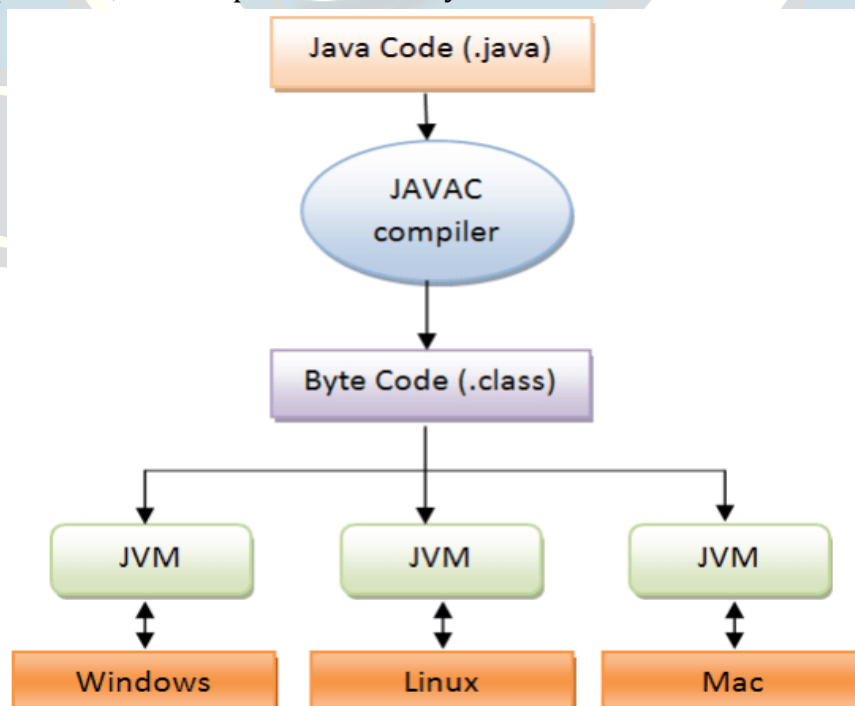
Byte code is a **highly optimized set of instructions** designed to be **executed by the JVM** which is the **interpreter** for the **byte code**. Byte code is not a machine specific code.

Byte code is a **universal code** and can be moved anywhere to any platform.

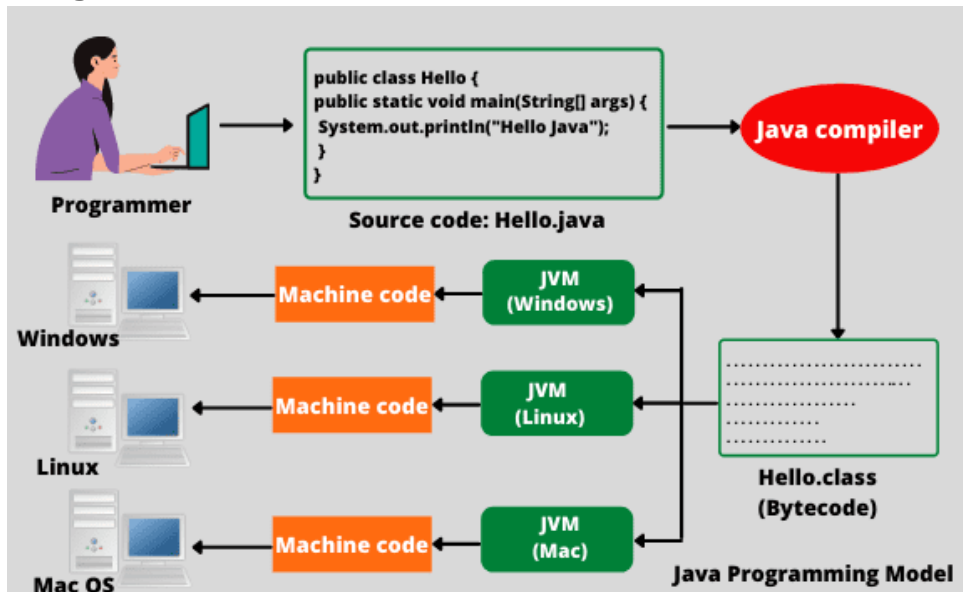
Therefore **java is portable**, as it can be carried to any platform.

JVM is a **virtual machine** which exists inside the **computer memory** and is a **simulated computer** within a computer which does all the **functions of a computer**.

Only the JVM needs to be implemented for each platform. Although the details of the JVM will defer from platform to platform, all interpret the same byte code.



Java Programming Model:-



3. Object-Oriented

Q] Why Java is called as Truly Object Oriented Programming?

[S-09, 11, 12, 13, W-11]

- Java is a true object-oriented language.
- Almost everything in Java is an object.
- All **program code and data** reside within objects and classes.
- Java includes all the features of OOP including **Inheritance, Polymorphism, and Dynamic Binding** etc.
- Most of **methods and constructors** used in Java are **overloaded**.
- So JAVA itself supports **Polymorphism**.
- JAVA does not support **Multiple Inheritance** to avoid duplication of data, but it is supported by the way of **Interface**.

4. Robust

Java is a robust language that can **handle run-time errors** as it **checks the code** during the **compile and runtime**.

Java incorporates the **concept of exception handling** where any serious **runtime error** is **identified** by the **JVM**, and it **will not be passed directly to the underlying system**.

Instead, it will immediately **terminate the program** and stop it from causing any harm to the system.

Java has a **strong memory management** system.

It also supports the concepts of **garbage collection** and exception handling.

5. Secure

Java is a secure language that ensures that **programs cannot access memory locations without authorization**. It has **access modifiers** to check **memory access**.

It **does not allow** programmers to **explicitly create pointers**.

Java also ensures that **no viruses enter an applet**.

Java's bytecode verifier **checks the code blocks** for any **illegal code** that violates the access right.

6. Distributed

Java is designed as a distributed language for creating applications on networks.

In Java, we can split a program into many parts and store these parts on different computers.

A Java programmer sitting on a machine can access another program running on the other machine. This feature in Java gives the advantage of distributed programming, which is very helpful when we develop large projects.

7. Simple, Small and Familiar

Java is a small and simple language. Many features of C and C++ that are **redundant** are not part of Java. For example, Java does not use **pointers, pre-processor header files, goto statement** and many others. It also eliminates **operator overloading** and **multiple inheritance**.

8. Multithreaded and Interactive

Multithreaded means handling multiple tasks simultaneously.

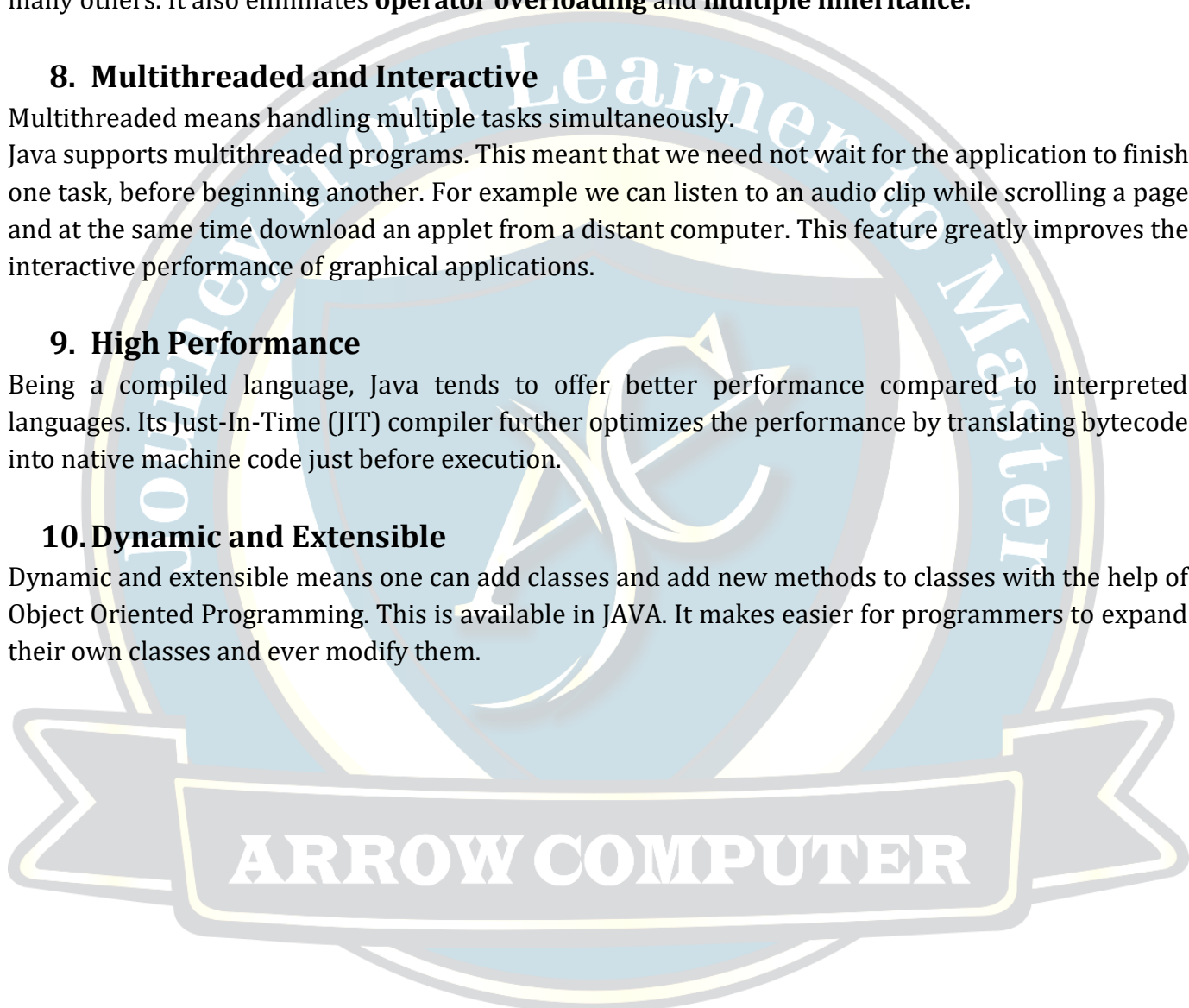
Java supports multithreaded programs. This meant that we need not wait for the application to finish one task, before beginning another. For example we can listen to an audio clip while scrolling a page and at the same time download an applet from a distant computer. This feature greatly improves the interactive performance of graphical applications.

9. High Performance

Being a compiled language, Java tends to offer better performance compared to interpreted languages. Its Just-In-Time (JIT) compiler further optimizes the performance by translating bytecode into native machine code just before execution.

10. Dynamic and Extensible

Dynamic and extensible means one can add classes and add new methods to classes with the help of Object Oriented Programming. This is available in JAVA. It makes easier for programmers to expand their own classes and ever modify them.



ARROW COMPUTER

Java Byte code and Java Virtual Machine

Q] What is byte code? Explain any two tools available in JDK.

[S-09, S-12]

Q] What is Byte code explain JVM?

[W-11, W-13]

Q] What is JVM? What is bytecode?

[S-13]

Compiler translates source code into machine code for specific computer (platform dependent). But the Java compiler produces an intermediate code known as byte code, for a machine that does not exist.

This machine is called as '**Java Virtual Machine**' (**JVM**) and it only exist inside the computer memory. It is simulated computer within the computer and performs all major functions of real computer.



Fig.: Process of Compilation

The output produced by the Java compiler is not executable code.

It is byte code, highly optimized set of instructions executed by java run time system or JVM. JVM is an interpreter for the byte code.

The virtual machine code generated by Java Interpreter is not machine specific (portable), but it acts as an intermediary between the virtual machine and real machine as shown in following Fig.



Fig. Process of converting byte code into machine code.

ARROW COMPUTER

Smallest individual units in a Program that are meaningful to the Java compiler are called Tokens. Java language includes 5 types of Tokens:

1. Keywords
2. Identifiers
3. Literal
4. Operators
5. Separators

For example, consider the following code.

```
public class Demo
{
    public static void main(String args[])
    {
        System.out.println("Arrow");
    }
}
```

In the above code, **public, class, Demo, {, static, void, main, (, String, args, [,],), System, ., out, println, Arrow**, etc. are the Java tokens.

1) Keywords:

- **pre-defined** reserved words of Language definition.
- Each keyword has a **special meaning**.
- **Must be written in Lowercase**
- Eg. do, while, int, if, short, void, continue, public etc.

abstract	assert	boolean
break	byte	case
catch	char	class
const	continue	default
do	double	else
enum	exports	extends
final	finally	float
for	goto	if
implements	import	instanceof
int	interface	long
module	native	new
open	opens	package
private	protected	provides
public	requires	return
short	static	strictfp
super	switch	synchronized
this	throw	throws
to	transient	transitive
try	uses	void
volatile	while	with

2) Identifiers:

- It usually defined by the user.
- Used for naming **classes, methods, variables, objects, packages** and **interface** in a program.
-
- Follows following Rules:
 1. Identifiers can only contain letters (a-z, A-Z), digits (0-9), the underscore (_), and the dollar sign (\$).
 2. The first character cannot be a digit. It must be a letter, an underscore, or a dollar sign.
 3. Java keywords (e.g., class, public, int, if, while, for, void) cannot be used as identifiers.
 4. Uppercase and Lowercase letters are distinct meaning that tot and TOT are treated as two distinct identifiers.
 5. Identifiers cannot contain spaces or other special characters (like #, @, or -).

3) Literals:-

Any constant value that can be assigned to the variable is called a literal.

```
int x = 100;           // Here 100 is a constant/literal.
```

Types of Literals:-

1. Integer
2. Character
3. Floating-point
4. String
5. Boolean
6. Null

4) Operators:-

An operator is a symbol that indicates one or more arguments and operates on them to produce result.

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Unary Operators
- Logical Operators
- Ternary Operators
- Bitwise Operators
- Shift Operators

5) Separators:-

- Separators are **symbols** used to indicate where **groups of code are divided and arranged**.
- Eg. {}, [], () etc.
- The separators in Java is also known as **punctuators**

➤ Variables, Constants and Data Types

Variables

Variables are the containers for storing the data values.

It is a **named memory location**.

Value of variable **may change** during the execution of the program.

Declaration of Variables:

Before a variable can be used, it must be declared.

Syntax:-

```
dataType variableName;
```

Example:

1. Declaration reserves a location in memory with a specific name and type.
2. It tells the compiler what the **variable name** is.
3. It specifies what **type data** the variable will **hold**.

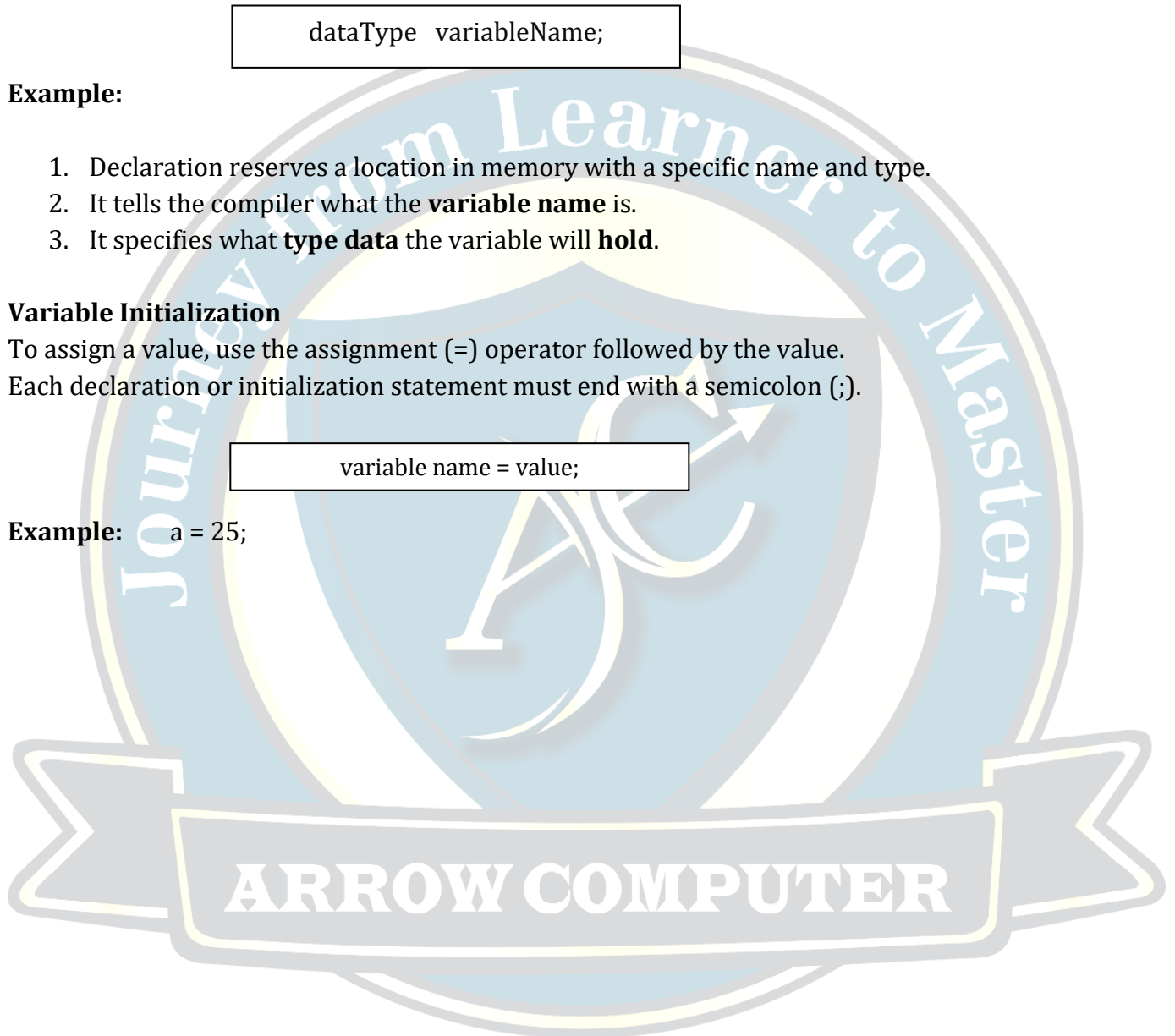
Variable Initialization

To assign a value, use the assignment (=) operator followed by the value.

Each declaration or initialization statement must end with a semicolon (;).

```
variable name = value;
```

Example: a = 25;



➤ Constants

Constants in Java refer to **fixed values** that **can not change** once assigned.

After declaration of symbolic constants, they should not be assigned any other values within the program using assignment operator.

Syntax:-

```
final data_type CONSTANT_NAME = value;
```

final:

The final keyword indicates that the value of the constant cannot be changed once it is initialized.

data_type:

The data type of the constant such as int, double, boolean, or String.

CONSTANT_NAME:

The name of the constant which should be written in all capital letters with underscores separating words.

value:

The initial value of the constant.

Example :-

```
final double PI = 3.14;
```

```
/* Calculate Area of Circle */
```

```
class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int radius;
```

```
        final double pi = 3.142;
```

```
        double area;
```

```
        radius = 5;
```

```
        area = pi * radius * radius;
```

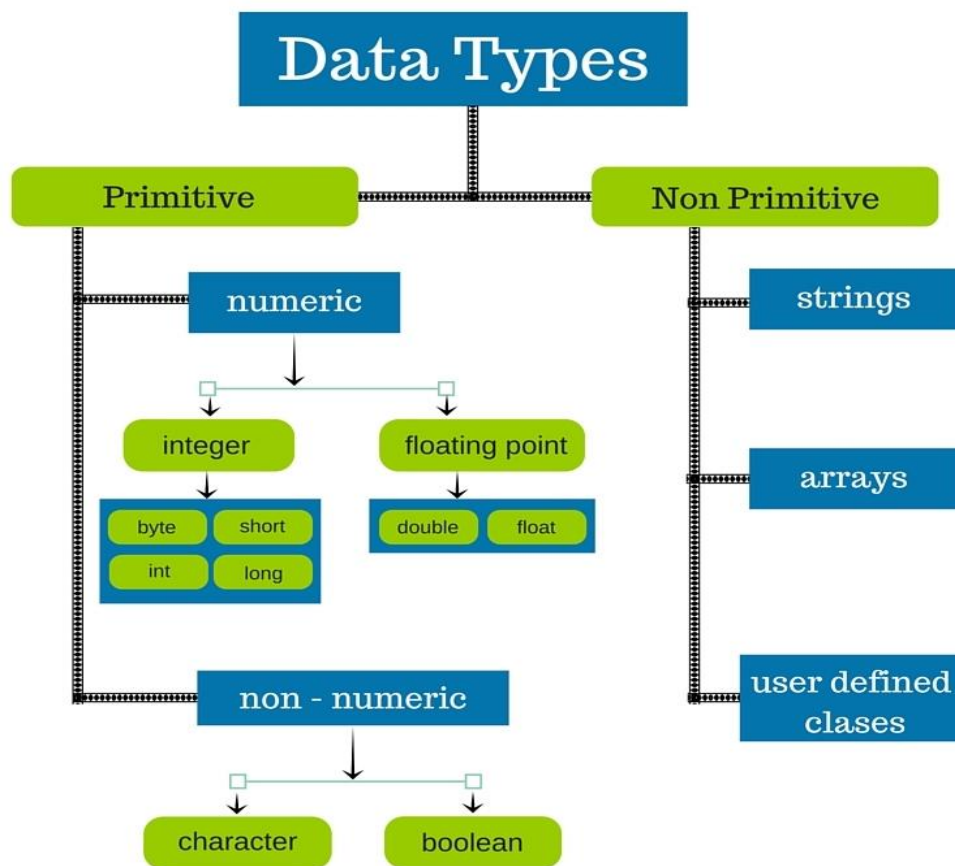
```
        System.out.println("Area of circle is :" + area);
```

```
    }
```

```
}
```

➤ Data Types

Q] Write all primitive data types available in Java with their storage sizes in bytes.
[W 09, S10, 12, 13, 17]



Data types in Java specify **how much memory is to be allocated** to Variable. Each variable has a data type that decides **what type of value the variable will hold**.

Primitive data types in Java are those data types whose variables can store **only one value** at a **time**. We cannot store multiple values of the same type. These data types are **pre-defined** in Java.

```
int x; // valid
x = 10; // valid because "x" store only one value at a time because it is the primitive type variable.
x = 10,20,30; // invalid.
```

Java defines eight primitive data types: boolean, char, byte, short, int, long, float, and double.

1] Integer Types

Integer type stores **whole numbers** that may be **positive** or **negative** and **should not contain** any **decimal places** such as 123, -96, and 5639.

Valid Integer types are **byte, short, int, and long**.

Long is used when int data type cannot hold a value.

It is 2 times larger than int(integer).

We must use **L** or **l** at the end of the value.

Example: 123L , 123l

Type	Size
byte	1 byte
short	2 byte
int	4 byte
long	8 byte

2] Floating Point Types

1. float :

The float data type in Java is a **single-precision 32-bit floating-point data type** used to store **fractional** numbers

A float uses 4 bytes (32 bits) of memory.

It offers a single precision of approximately **6 to 7 decimal digits**.

When assigning a literal value to a float variable, you must append the letter **f** or **F** to the number (e.g., 3.14f), **because Java treats all floating-point literals as double by default**.

```
float weight = 70.645f;  
or  
float weight = 70.645F;
```

Type	Size
float	4 byte
double	8 byte

2. double:

The double keyword in Java is also a primitive data type, and its size limit is **8 byte or 64 bits double-precision**, up to **15 digits** precision after the decimal.

We often use this keyword when we have a **larger floating value** and if we want a more **precise** and **accurate** value.

By default, Java considers any decimal or float values of double type, so we do not need to typecast any decimal value to double manually.

Hence, adding the suffix '**d**' or '**D**' in the number is optional in the case of Double data type values.

```
double weight = 70.6458763;  
or  
double weight = 70.6458763d;  
or  
double weight = 70.6458763D;
```

Note:- double is best suited for applications that require a high degree of accuracy and a wide range of values

3] Character Type

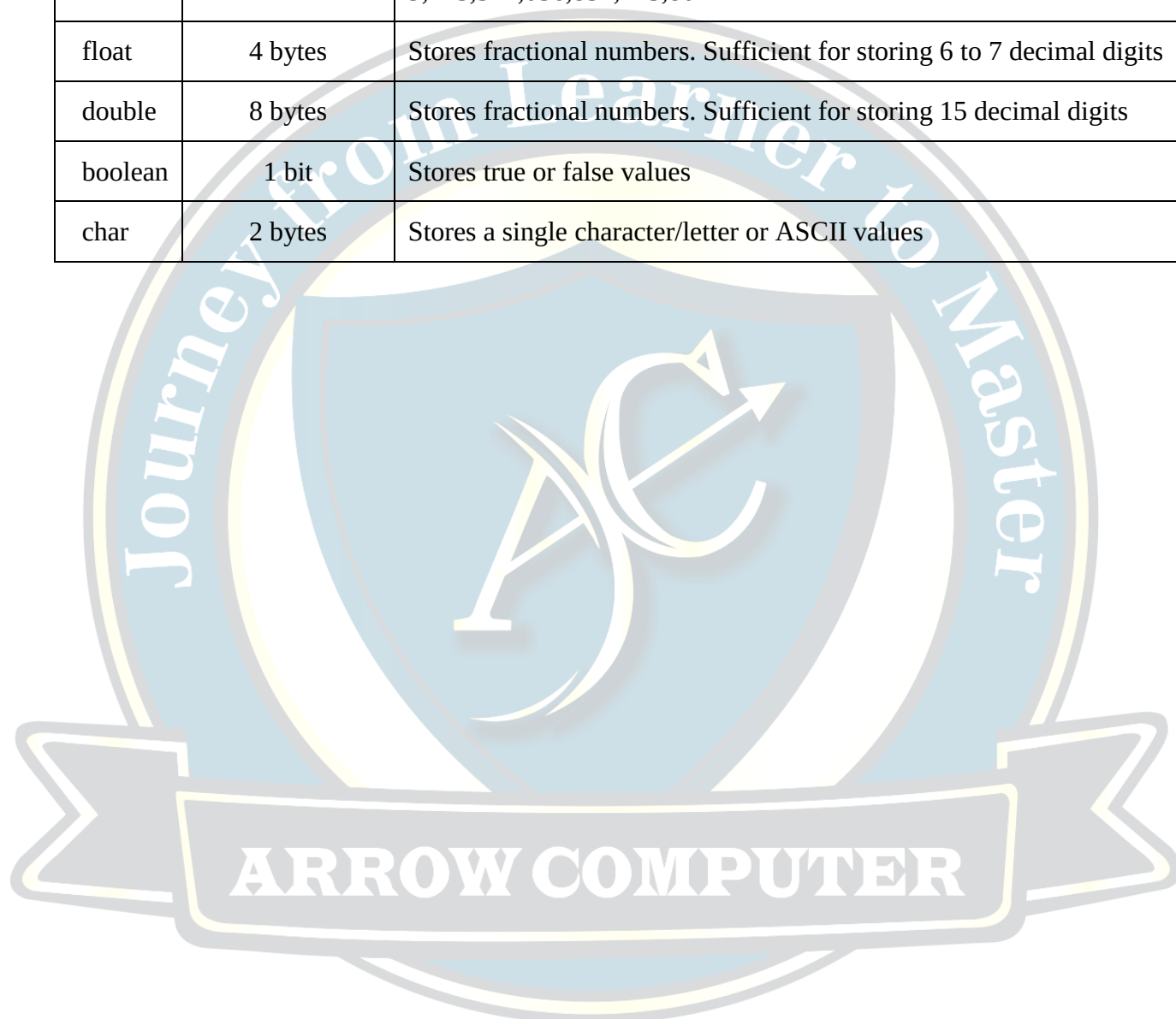
In order to store character constant in memory, Java provides a character data type called char.

The char type assumes a size of **2 bytes** but basically, it can hold a **single character**.

4] Boolean Type

Used when we want to test a particular condition during the execution of the program. There are only two values that a Boolean type can take: **true and false**. It uses only **1 bit**.

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values



➤ Operators

An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations. Java operators:

- 1) Arithmetic
- 2) Relational
- 3) Logical
- 4) Assignment
- 5) Increment and Decrement
- 6) Conditional
- 7) Bitwise
- 8) Special

1) Arithmetic Operators

Q] Explain Arithmetic Operators with suitable example.

[W-08, S-13]

Arithmetic Operators are used to construct mathematical expressions.

Operators	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo division (Remainder)

2) Relational Operator

Q] Explain Relational Operators with suitable example.

[S-10, W-11]

Relational Operators are used to compare two quantities and depending on their relation, take certain decision. There are 6 relational operators in JAVA.

The relational operators in java are:

1) < Less than Operator

This operator is used to check the inequality of two expressions. It returns true if the first expression is less than the second expression else returns false.

eg. if $x=10$; then $x < 20$ is true
 $20 < x$ is false

2) > Greater than operator

This operator is also used to check the inequality of two expressions. It returns true if the first expression is greater than the second one else returns false.

eg. if $x=10$; then $20 > x$ is true
 $20 < x$ is false

3) <= Less than or equal to

This operator returns true if the first expression is less than or equal to the second expression else returns false.

eg. if $x=10$; then $x \leq 20$ is true
 $20 \leq x$ is false

4) >= Greater than or equal to

This operator returns true if the first expression is greater than or equal to the second expression else returns false.

eg. if x=10; then 20>=x is true
x>=20 is false

5) == Is equal to

This operator returns true if the values of both the expressions are equal else returns false.

eg. if x=10; then x==20 is false

6) != Not equal to

This operator returns true if the values of both the expressions are not equal else returns false.

eg. if x=10; then x!=20 is true

Example of Relational Operator:-

```
class RelationalOperators {  
public static void main(String args[])  
{  
int a = 10, b = 20;  
System.out.println("a is " + a);  
System.out.println("b is " + b );  
System.out.println("a == b is " + (a == b) );  
System.out.println("a != b is " + (a != b) );  
System.out.println("a > b is " + (a > b) );  
System.out.println("a < b is " + (a < b) );  
System.out.println("a >=b is " + (a >= b) );  
System.out.println("a <=b is " + (a <= b) );  
}  
}
```

Output is:

```
a == b is false  
a != b is false  
a > b is false  
a < b is true  
a >=b is false  
a <=b is true
```

3) Logical Operators

Q] Explain Logical Operators with suitable example.

[S-10,S-15, W-11, 13]

Logical Operators are used when we want to form compound conditions by combining two or more relations. Java has three logical operators as shown in table.

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

Example:-

a > b && x==10

This expression which combines two or more relational expressions is known as a logical expression or a compound relational expression. The logical relation given above is True only if both a > b and x==10 are true. If either (or both) of them are false, the expression is false.

Truth Table of Logical Operators

OP-1	OP-2	Value of the expression		! OP-1
		op-1 && op-2	op-1 op-2	
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

OP-1 && OP-2 is true if both the operands are true and false otherwise.

OP-1 || OP-2 is true if either one of the operands are true.

Example of Logical Operator

```
class Log
{
    public static void main(String args[])
    {
        boolean A=true;
        boolean B=false;
        System.out.println("A is " +A);
        System.out.println("B is " +B);
        System.out.println("A || B is " +(A||B));
        System.out.println("A && B is " +(A&&B));
        System.out.println("!A is " +(!A));
        System.out.println("(A||B)&&A is "+((A||B)&A));
    }
}
```

Output:

```
A is true
B is false
A || B is true
A && B is false
! A is false
(A || B) && A is true
```

4) Bit-Wise Operators

Q] Explain any two bit-wise operators with example. [W-15]

1) Bitwise NOT (~): called bitwise complement, the unary NOT operator, inverts all of the bits of its operand.

e.g Let value of X is 6;

Its binary Equivalent is :- 0000 0110

Using Not operator (~X) :- 1111 1001 // This is 1's complement of X

Here the Leftmost digit is 1.

That is the number is negative.

And negative nos are stored as 2's Complement.

So in order to find decimal equivalent of result, we need to find 2's Complement. (1's Complement +1).

1111 1001 -> 0000 0110 + 1 -> 0000 0111 (It is equivalent to 7)

Result -> 1's Complement -> 2's Complement

Since the MSB was 1, as mentioned above, therefore result is :- (-7)

2) Bitwise AND (&) :

The AND operator, &, produce a 1 bit if both operands are also 1. A zero is produced in all the cases.

e.g 0101 & 0011 = 0001
(decimal 5) (decimal 3) (decimal 1)

3) Bitwise OR (|) :

The OR operator, |, combines bits such that if either of the bits in the operand is a 1, then the resultant bit is a 1.

e.g 0101 | 0011 = 0111
(decimal 5) (decimal 3) (decimal 7)

4) Bitwise XOR (^) :

The XOR operator, ^, combines bits such that if exactly one operand is 1, then the result is 1. Otherwise result is zero.

e.g 0101 ^ 0011 = 0110
(decimal 5) (decimal 3) (decimal 6)

5) The Left Shift (<<):

The left shift operator, <<, shifts all of the bits in a value, to the left a specified number of times specified by num.

General form: value <<num

e.g. int x=12;

x << 2 ;

0000 1100 << 2 = 0011 0000 (decimal 48)

6) The Right Shift (>>):

The right shift operator, >>, shifts all of the bits in a value, to the right a specified number of times specified by num.

General form: value >> num.

e.g. (x=32)

x >> 2;

0010 0000 >> 2 = 0000 1000 (decimal 8)

Program of bitwise operators

class Bitwise

```
{  
    public static void main(String[] args)  
    {  
        int A = 35, B = 2;  
        System.out.println("A&B is " + (A & B));  
        System.out.println("A|B is " + (A | B));  
        System.out.println("A^B is " + (A ^ B));  
        System.out.println("~A is " + (~A));  
        System.out.println("A<<B is " + (A << B));  
        System.out.println("A>>B is " + (A >> B));  
    }  
}
```

Output:-

A&B is 2

A|B is 35

A^B is 33

~A is -36

A<<B is 140

A>>B is 8

A	B	A&B	A B	A^B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

5) Increment and Decrement Operators

Java has two very useful operators i.e. increment (++) and decrement (--) operators.

The **increment** operator **increases its operand by one**.

The **decrement** operator **decreases its operand by one**.

This statement: `x = x + 1;` can be rewritten like `x++;`

Similarly, this statement: `x = x - 1;` is equivalent to `x--;`

Both are unary operators and are used in following form:

- 1) Postfix Increment (x++)
- 2) Prefix Increment (++x)
- 3) Postfix Decrement (x--)
- 4) Prefix Decrement (--x)

In the **prefix** form, the **operand** is **incremented** or **decremented** before the value is assigned to the other variable in the expression.

For example:

```
x = 42;  
y = ++x;
```

In above example; first value of x is incremented (43) and this changed value is assigned to variable y (43).

In the **postfix** form, first the **operand** is **assigned** to another variable and then it is incremented or decremented.

For example:

```
x = 42;  
y = x++;
```

In the above example, first value of x is assigned to variable y (42) and then the value of x is incremented(43).

Program of increment operator

```
class IncDec  
{  
    public static void main(String args[])  
    {  
        int a = 10;  
        int b = 5;  
        int c;  
        int d;  
        System.out.println("Initial Value of a is " + a);  
        System.out.println("Initial Value of b is " + b);  
        c = ++a;  
        System.out.println("a = " + a);  
        System.out.println("c = " + c);  
        d = b++;  
    }  
}
```

```
System.out.println("b = " + b);  
System.out.println("d = " + d);  
c = --a;  
System.out.println("c = " + c);  
System.out.println("a = " + a);  
d = b--;  
System.out.println("b = " + b);  
System.out.println("d = " + d);  
}  
}
```

Output:

```
a = 11  
c = 11  
b = 6  
d = 5  
c = 10  
a = 10  
b = 5  
d = 6
```



6) Conditional Operator

The conditional operator is also known as the ternary operator.

This operator consists of **three operands** and is used to evaluate Boolean expressions.

It can replace certain types of if-then-else statements.

General form of Conditional Operator (?:) is

variable x = **expression1 ? expression2 : expression3;**

Here, expression1 can be any expression that evaluates to a **boolean** value.

If expression1 is **true**, then **expression2** is **evaluated** and **its value will be the value of conditional expression**.

If **expression1** is **false**, then **expression3** is **evaluated** and **its value will be the value of conditional expression**.

Both expression2 and expression3 are required to return the same type, which can't be **void**.

Example:

a = (x > y) ? x : y ;

When Java evaluates this assignment expression, it first looks at the expression to the left of the question mark.

If **x > y**, then the expression between the question mark and the colon is evaluated and used as the value of the entire ? expression.

If **x < y**, then the expression after the colon is evaluated and used for the value of the entire ? expression. The result produced by the ? operator is then assigned to **a**.

Example 1: Program of the ?: operator

```
class Ternary
{
    public static void main(String args[])
    {
        int a=10;
        int b=20;
        int c;
        c=(a>b)?a:b;
        System.out.println("Largest number is \t"+c);
    }
}
```

➤ Scope of Variable

The area of program where the variable is accessible is called its Scope.

The variable scope also refers to its lifetime.

A block of code refers to all of the code between curly braces {}.

A variable declared inside pair of brackets "{" and "}" in a method has scope within the brackets only and it is only accessible by the code between the curly braces,

```
// demonstrate block scope.
class Scope
{
public static void main (String args[])
{
int n1;                // Visible in main
n1 = 10;
if(n1==10)
{
// start new scope
int n2 = 20;           // visible only to this block
// n1 and n2 both visible here.
System.out.println("n1 and n2 : " + n1 + " " + n2);
}
//n2 = 100;            // Error! N2 not known here
//n1 is still visible here.
System.out.println("n1 is " + n1);
}
}
```

Output:

```
n1 and n2 : 10 20
n1 is 10
```

n1 is declared in main block thus it is accessible in main block.

n2 is declared in if block thus it is only accessible inside if block

Any attempt to access it outside block will cause compilation error.

Nested Block can have access to its outmost block.

if block is written inside main block thus all the variables declared inside main block are accessible in if block

➤ Types of JAVA Variables

Java variables are classified into three kinds:

- instance variables
- class variables / Static Variable
- local variables

1) Instance variable

Instance variables are declared in a class, but outside a method, constructor or any block.

Instance variables are **created when an object is created** with the use of the **keyword 'new'** and **destroyed when the object is destroyed**.

Hence they are associated with the objects.

They take different values for each object.

Access modifiers can be given for instance variables.

Instance variables have default values. For numbers, the default value is 0, for Booleans it is false.

2) Class Variables / Static variable

Class variables also known as static variables are declared with the **static** keyword in a class, but outside a method, constructor or a block.

You can **create a single copy** of the static variable and **share it among all the objects of the class**.

3) Local Variable

Local variables are declared in methods, constructors, or blocks.

Local variables are created when the method, constructor or block is entered and the variable will be destroyed once it exits the method, constructor, or block.

Access modifiers cannot be used for local variables.

Local variables are visible only within the declared method, constructor, or block.

There is no default value for local variables, so local variables should be declared and an initial value should be assigned before the first use.

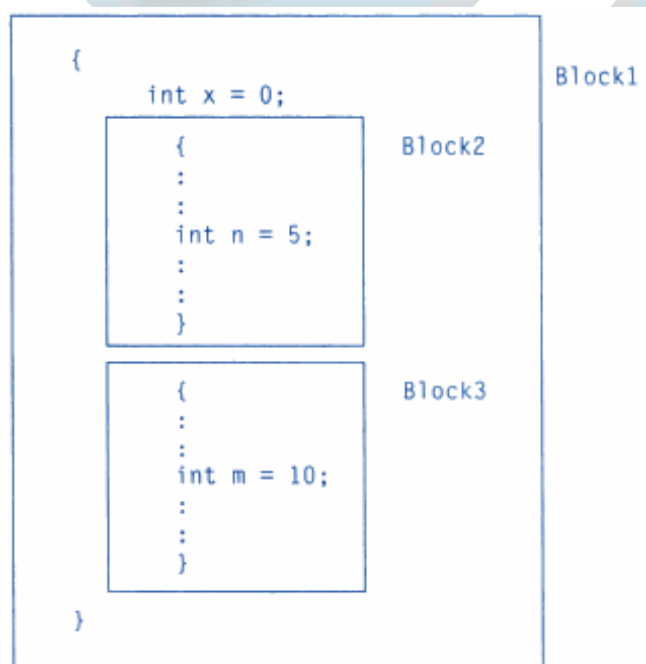


Fig: Nested Program Block.

Each block contains its own set of local variable declaration.

We cannot declare variable to have same name as one in an outer block.

Here variable x declared in Block 1 is available in all three blocks.

However variable n is available in block 2 only, because it goes out of the scope at the end of block 2.

Similarly, m is accessible in block 3 only.



➤ Type Casting

Q] Explain 'Type Casting' with suitable example.

[S-11]

Q] What do you mean by 'Type Casting'? When is it needed?

[W-12, 14]

Assigning a value of one type of a variable to another type is known as Type Casting

There are 2 types of type casting

1. Widening or Implicit type casting
2. Narrowing or Explicit type casting

1. Implicitly Type Casting / Automatic Type Conversion/ Widening casting:

Widening casting is done automatically when passing a **smaller size type to a larger size type**.

For assigning one type of data to another type of data variables, **casting** is required.

But java **provides this type of assignment** without **casting**, known as **Automatic Type Conversion**.

It is possible only if the destination type has enough precision to store the source value.

Example: **int** is large enough to hold a **byte** value.

Hence

```
byte b=75;
```

```
int a = b;
```

are valid statements.

Example:

```
public class Test
{
    public static void main(String args[])
    {
        int i = 100;
        long l = i; // no explicit type casting require
        float f = l; // no explicit type casting required
        System.out.println ("Int value " + i);
        System.out.println ("Long value " + l);
        System.out.println ("Float value " + f);
    }
}
```

2. Explicit Type casting/ Narrowing Casting

When you are assigning a larger type value to a variable of smaller type, then you need to perform explicit type casting.

Narrowing casting must be done **manually** by placing the type in parentheses in front of the value. Casting to smaller type can result in a loss of data.

The syntax is:

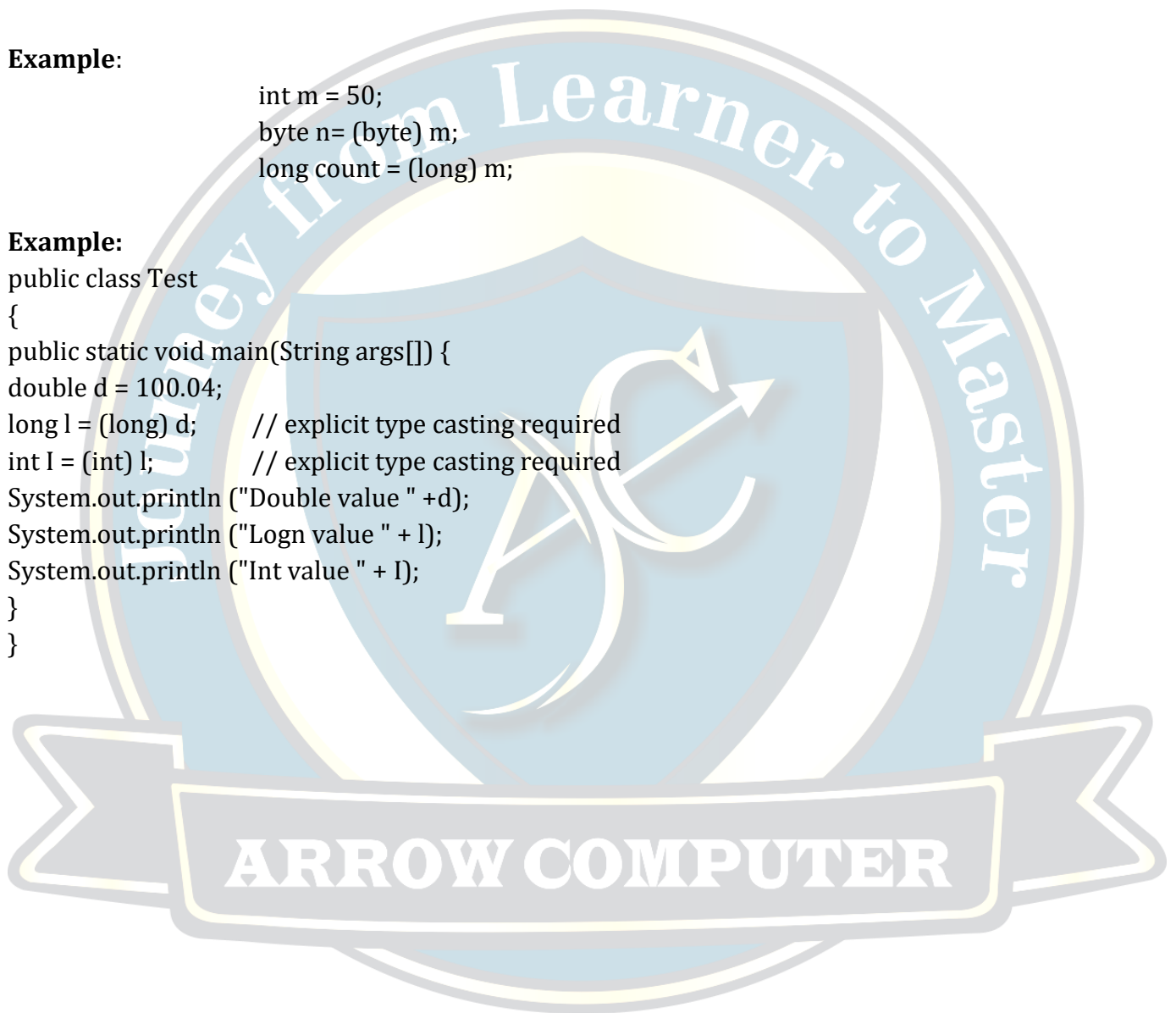
```
type variable = (type) variable1
```

Example:

```
int m = 50;
byte n= (byte) m;
long count = (long) m;
```

Example:

```
public class Test
{
    public static void main(String args[]) {
        double d = 100.04;
        long l = (long) d;    // explicit type casting required
        int I = (int) l;      // explicit type casting required
        System.out.println ("Double value " +d);
        System.out.println ("Logn value " + l);
        System.out.println ("Int value " + I);
    }
}
```



Java User Input

The **Scanner** class is used to get **user input**, and it is found in the **java.util** package.

To use the **Scanner** class, create an **object** of the class.

To create an object of **Scanner** class, we **usually pass the predefined** object **System.in**, which represents the **standard input stream**.

Then we can use the **nextLine()** method, which is **used to read Strings**:

```
import java.util. Scanner; // Import the Scanner class
class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in); // Create a Scanner object
        System.out.println("Enter username");
        String userName = sc.nextLine(); // Read user input
        System.out.println("Username is: " + userName); // Output user input
    }
}
```

Input Types

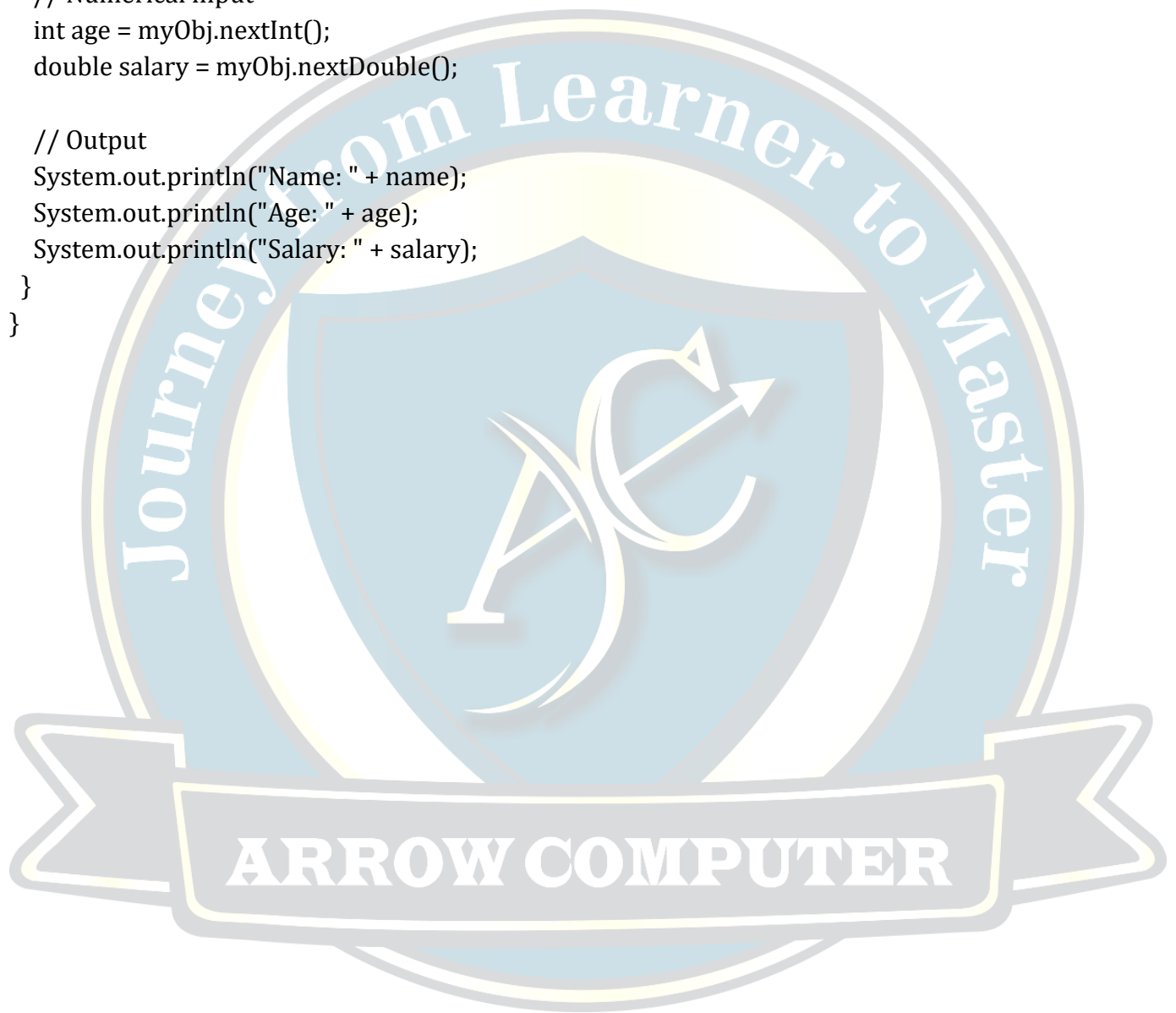
In the example above, we used the **nextLine()** method, which is used to read Strings. To read other types, look at the table below:

Method	Description
nextBoolean()	Reads a boolean value from the user
nextByte()	Reads a byte value from the user
nextDouble()	Reads a double value from the user
nextFloat()	Reads a float value from the user
nextInt()	Reads a int value from the user
nextLine()	Reads a String value from the user
nextLong()	Reads a long value from the user
nextShort()	Reads a short value from the user

In the example below, we use different methods to read data of various types:

```
import java.util.Scanner;
```

```
class ScannerDemo {  
    public static void main(String[] args)  
    {  
        Scanner myObj = new Scanner(System.in);  
        System.out.println("Enter name, age and salary:");  
  
        // String input  
        String name = myObj.nextLine();  
  
        // Numerical input  
        int age = myObj.nextInt();  
        double salary = myObj.nextDouble();  
  
        // Output  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("Salary: " + salary);  
    }  
}
```



➤ Decision Making and Looping

Decision Making with if statement

Q] Write general syntax of any two decision making statements and also give its examples. (Any two decision making statement & example - 2 marks each) [Note: Any relevant example can be considered**]**

Ans:

The if statement is powerful decision making statement & is used to control flow of execution of statements.

The if statement is the simplest one in decision statement. The if keyword is followed by test expression in parentheses.

It is basically a two way decision statement & is used in conjunction with an expression. It has following syntax:

if (test expression)

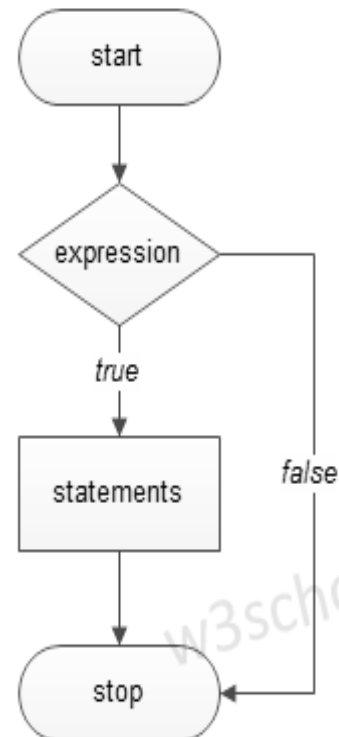
It allows computer to evaluate expression first & then depending on value of expression (relation or condition) is "true" or "false", it transfers control to a particular statement. The program has two parts to follow one for "if condition is true" & the other for "if condition is false"

1) Simple if Statement

General form of a statement is

```
if (test expression)
{
    statement block;
}
statement n;
```

Statement in block may be single statement or multiple statements. If condition is true then statement block will be executed otherwise block is skipped & statement n is executed.



Example of if statement

```
public class Sample
{
    public static void main(String args[])
    {
        int a=20, b=30;
        if(b>a)
            System.out.println("b is greater");
    }
}
```

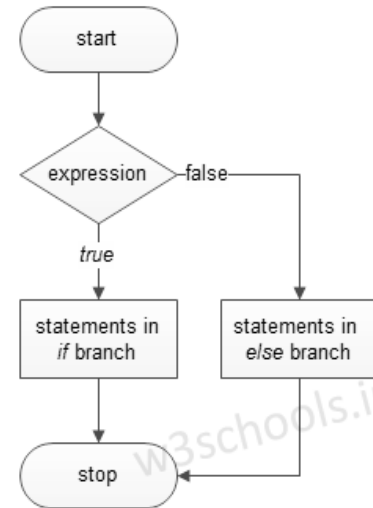
Output:

b is greater

The if.....else Statement

General form of a statement is

```
if (test expression)
{
    true block statements;
}
else
{
    false block statements;
}
statement n;
```



If test expression is true, then true block statement (s) following the if statement are executed otherwise, false-block statement(s) are executed. In both cases statement n will always be executed.

Example of if else statements

```
public class Sample
{
    public static void main(String args[]) {
        int a = 80, b = 30;

        if (b > a)
        {
            System.out.println("b is greater");
        }
        else
        {
            System.out.println("a is greater");
        }
    }
}
```

Even Odd Number Program:-

```
import java.util.*;
public class Main
{
    public static void main (String[] args)
    {

        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number");
        int num = sc.nextInt();
        if(num%2==0)
        {
            System.out.println("Number is Even");
        }
        else
        {
            System.out.println("Number is Odd");
        }
    }
}
```

Nested if else statements

When series of decision are involved, more than one if else statements are used in nested.

General form is:-

```
if (test expression 1)
{
    if (test expression 2)
    {
        statement block 1;
    }
    else
    {
        statement block 2;
    }
}
else
{
    statement block 3;
}

statement n;
```

If condition-1 is false, statement block-3 will be executed; otherwise it continues to perform second test. If condition-2 true, the statement block-1 will be evaluated; otherwise statement block-2 will be evaluated & then control is transferred to statement n.

Example

```
class Test
{
public static void main(String[] args)
{
    //Creating two variables for age and weight
    int age=15;
    int weight=48;
    //applying condition on age and weight
    if(age>=18)
    {
        if(weight>=50)
        {
            System.out.println("You are eligible to donate blood");
        }
        else
        {
            System.out.println("Not eligible to donate blood : Weight is Less than 50");
        }
    }
    else
    {
        System.out.println("Not eligible to donate blood :Age must be greater than 18");
    }
}
}
```

Example :

```
class Test {
    public static void main(String[] args) {
        int n1 = 150, n2 = 180, n3 = 170;

        if (n1 >= n2) {
            if (n1 >= n3)
                System.out.println("Student with height: " + n1 + " is the tallest.");
            else
                System.out.println("Student with height: " + n3 + " is the tallest.");
        } else {
            if (n2 >= n3)
                System.out.println("Student with height: " + n2 + " is the tallest.");
            else
                System.out.println("Student with height: " + n3 + " is the tallest.");
        }
        System.out.println("\n");
    }
}
```

The else.....if ladder

When multipath decisions are involved either nested if or elseif ladder can be used. A multipath decision is chain of ifs in which statement associated with each else is if.

General form is:

```
if (test expression 1)
    statement 1;
else if (test expression 2)
    statement 2;
else if (test expression 3)
    statement 3;
.
.
.
else if (test expression n)
    statement n;
else
    default statement;
statement x;
```

Example of if else ladder

```
import java.util.*;
public class Main
{
    public static void main (String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter Two numbers");
        int num1 = sc.nextInt();
        int num2 = sc.nextInt();
        if(num1>num2)
        {
            System.out.println(num1+" is greater than "+num2);
        }
        else if(num2>num1)
        {
            System.out.println(num2+" is greater than "+num1);
        }
        else
        {
            System.out.println(num1+" is equal to "+num2);
        }
    }
}
```

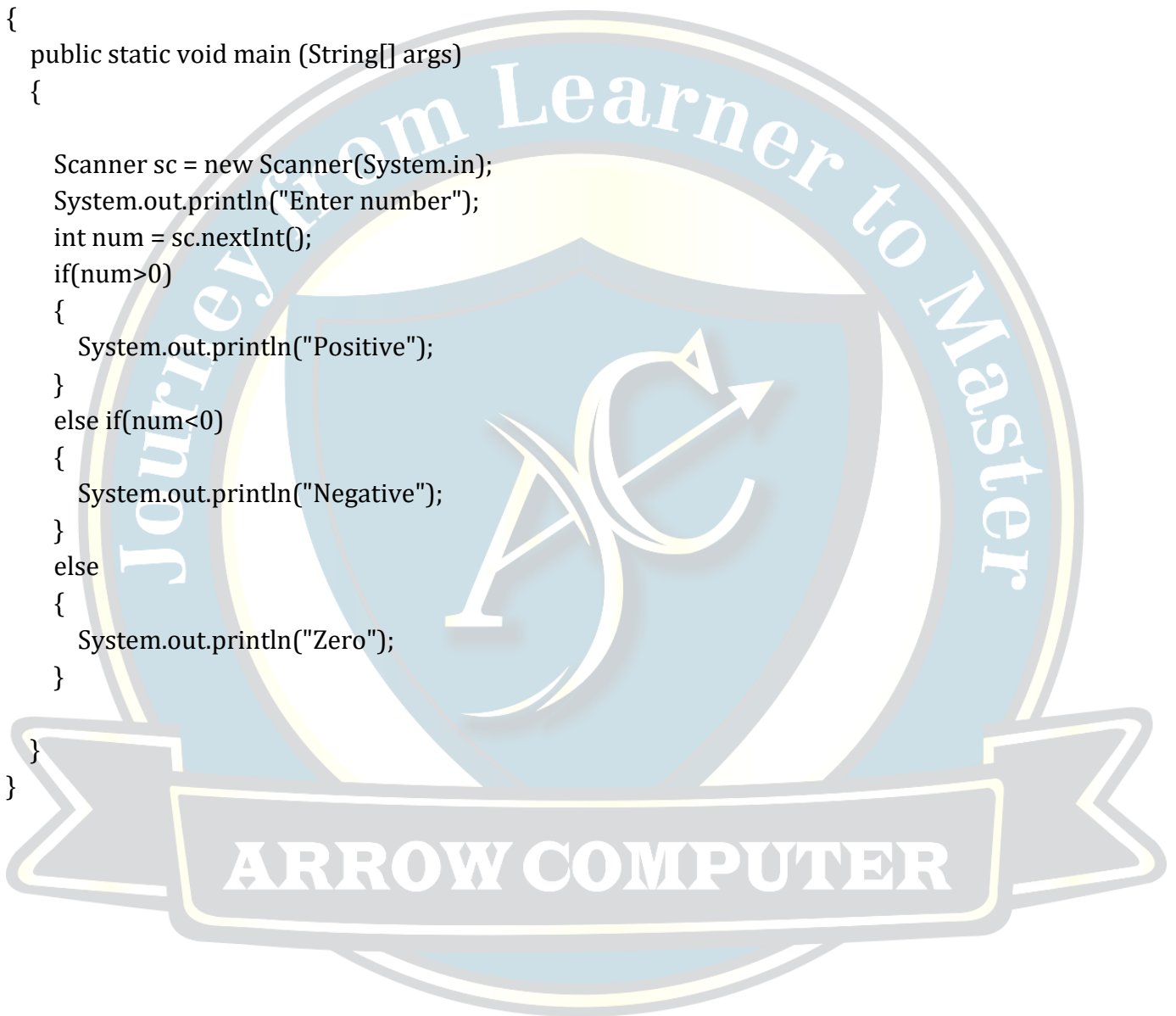
Output:

Both are equal

This construction is known as else.....if ladder. The conditions are evaluated from top (of the ladder) to downwards. As soon as true conditions are found, statement associated with it is executed & control is transferred to statement-x (skipping the rest of the ladder). When all the n conditions becomes false, then final else containing default statement will be executed.

Example 1:

```
import java.util.*;
public class Main
{
    public static void main (String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number");
        int num = sc.nextInt();
        if(num>0)
        {
            System.out.println("Positive");
        }
        else if(num<0)
        {
            System.out.println("Negative");
        }
        else
        {
            System.out.println("Zero");
        }
    }
}
```



Example 2:

```
class Test
```

```
{  
    public static void main(String[] args)  
    {  
        int m1,m2,m3,m4,m5,tot,per;  
        m1=32;  
        m2=35;  
        m3=40;  
        m4=43;  
        m5=51;  
        tot = m1+m2+m3+m4+m5;  
        per=tot/5;  
        System.out.println("Total is "+tot);  
        System.out.println("Percentage is "+per);  
        if(per>=75 && m1>=40 && m2>=40 && m3>=40 && m4>=40 && m5>=40)  
        {  
            System.out.println("Distinction");  
        }  
        else if(per>=60 && m1>=40 && m2>=40 && m3>=40 && m4>=40 && m5>=40)  
        {  
            System.out.println("First Class");  
        }  
        else if(per>=50 && m1>=40 && m2>=40 && m3>=40 && m4>=40 && m5>=40)  
        {  
            System.out.println("Second Class");  
        }  
        else if(per>=40 && m1>=40 && m2>=40 && m3>=40 && m4>=40 && m5>=40)  
        {  
            System.out.println("Pass");  
        }  
        else  
        {  
            System.out.println("Fail");  
        }  
    }  
}
```

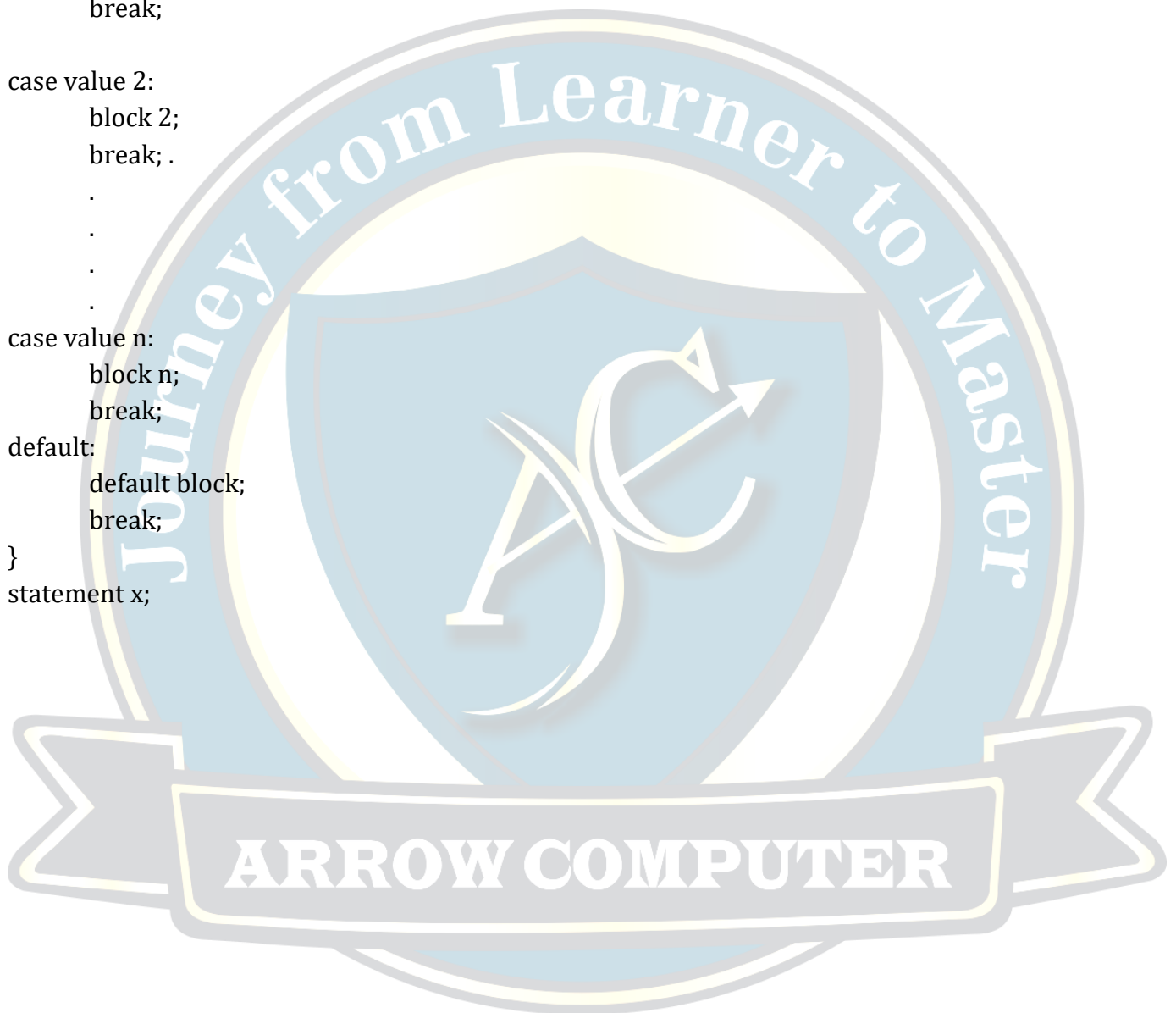

Switch Statement

The Java *switch statement* executes one statement from multiple conditions. It is like if-else-if ladder statement.

General form:

```
switch (expression)
{
    case value 1:
        block 1;
        break;

    case value 2:
        block 2;
        break;
    .
    .
    .
    case value n:
        block n;
        break;
    default:
        default block;
        break;
}
statement x;
```



Example:

```
class Switch
{
public static void main(String[] args) {
int day = 4;
switch (day) {
case 1:
    System.out.println("Monday");
    break;
case 2:
    System.out.println("Tuesday");
    break;
case 3:
    System.out.println("Wednesday");
    break;
case 4:
    System.out.println("Thursday");
    break;
case 5:
    System.out.println("Friday");
    break;
case 6:
    System.out.println("Saturday");
    break;
case 7:
    System.out.println("Sunday");
    break;
}
}
}
// Outputs "Thursday" (day 4)
```

The expression is an integer expression or character.

value1, value2.... valuen are constants or constant expressions which must be evaluated to integral constants are known as case labels.

Each of these values should be unique within switch statement.

Block1, block 2 are statement lists & may contain zero or more statements. There is no need to put braces around these blocks but case labels must end with colon (:).

When switch is executed, value of expression is successively compared against values value-1, value-2.

If case is found whose value matches with value of expression, then block statements that follows case are executed.

The **break statement** at end of each block signals end of particular case & causes exit from switch statement & transferring control to statement following switch.

The default is optional case. When present it will be executed if value of expression does not match with any of cases values.

If not present, no action takes place when all matches fail & control goes to statement x.

Note that if the default statement is used as the last statement in a switch block, it does not need a break.

But if its not last case in Switch, then there must be a break statement.

```
class Switch
{
public static void main(String[] args)
{
    String str = "two";

    switch (str)
    {
    case "one":
    System.out.println("one");
    break;
    case "two":
    System.out.println("two");
    break;

    case "three":
    System.out.println("three");
    break;
    default:

    System.out.println("no match");
    }
    }
}

class Switch
{
    public static void main(String[] args)
    {
        char ch = '*';
        int a=20,b=30;
        switch(ch)
        {
        case '+':
            System.out.println(a+b);
            break;
        case '-':
            System.out.println(a-b);
            break;
        case '*':
            System.out.println(a*b);
```

```

        break;
    case '/':
        System.out.println(a/b);
        break;
    default:
        System.out.println("Wrong Choice");
    }
}

```

Examples:

```

import java.io.*;
class Switch
{
    public static void main(String args[]) throws Exception
    {
        int choice,num1;
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        System.out.println("1. Odd / Even");
        System.out.println("2. Square");
        System.out.println("Enter the choice");
        choice=Integer.parseInt(br.readLine());
        switch (choice)
        {
            case 1:
                System.out.println("Enter the number");
                num1=Integer.parseInt(br.readLine());
                if(num1%2==0)
                {
                    System.out.println("Number is Even");
                }
                else
                {
                    System.out.println("Number is Odd");
                }
                break;
            case 2:
                System.out.println("Enter the number");
                num1=Integer.parseInt(br.readLine());
                System.out.println("Square of the number is" +(num1*num1));
                break;
            default:
                System.out.println("Enter the correct choice");
        }
    }
}

```

Loops in Java

In programming languages, loops are used to execute a set of instructions/functions repeatedly when some conditions become true. There are three types of loops in java.

- for loop
- while loop
- do-while loop

Java For Loop vs While Loop vs Do While Loop

Comparison	for loop	while loop	do while loop
Introduction	The Java for loop is a control flow statement that iterates a part of the programs multiple times.	The Java while loop is a control flow statement that executes a part of the programs repeatedly on the basis of given boolean condition.	The Java do while loop is a control flow statement that executes a part of the programs at least once and the further execution depends upon the given boolean condition.
When to use	If the number of iteration is fixed, it is recommended to use for loop.	If the number of iteration is not fixed, it is recommended to use while loop.	If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use the do while loop.
Syntax	<pre>for(init;condition;incr/decr) { // code to be executed }</pre>	<pre>while(condition) { //code to be executed }</pre>	<pre>do { //code to be executed }while(condition);</pre>
Example	<pre>//for loop for(int i=1;i<=10;i++) { System.out.println(i); }</pre>	<pre>//while loop int i=1; while(i<=10){ System.out.println(i); i++; }</pre>	<pre>//do-while loop int i=1; do{ System.out.println(i); i++; }while(i<=10);</pre>
Syntax for infinitive loop	<pre>for(; ;) { //code to be executed }</pre>	<pre>while(true){ //code to be executed }</pre>	<pre>do{ //code to be executed }while(true);</pre>

A] Java For Loop

The Java for loop is used to iterate a part of the program several times. If the number of iteration is fixed, it is recommended to use for loop. In a simple for we can initialize the variable, check condition and increment/decrement value. It consists of four parts:

Initialization: It is the initial condition which is executed once when the loop starts. Here, we can initialize the variable, or we can use an already initialized variable. It is an optional condition.

Condition: It is the second condition which is executed each time to test the condition of the loop. It continues execution until the condition is false. It must return boolean value either true or false. It is an optional condition.

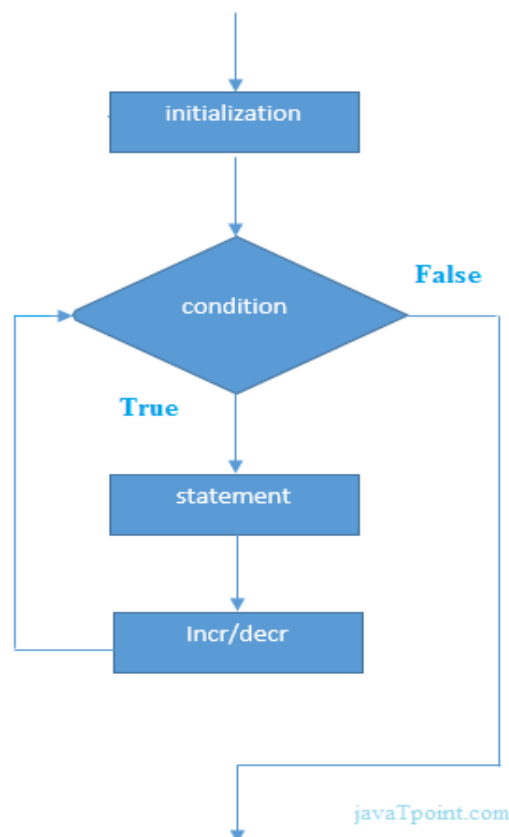
Statement: The statement of the loop is executed each time until the second condition is false.

Increment/Decrement: It increments or decrements the variable value. It is an optional condition.

Syntax:

```
for(initialization; condition; incr/decr)
{
//statement or code to be executed
}
```

Flowchart:



for loop in java flowchart

Example:

```
public class ForExample
{
    public static void main(String[] args)
    {
        //Code of Java for loop
        for(int i=1;i<=10;i++){
            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```



Java Nested For Loop

If we have a for loop inside the another loop, it is known as nested for loop. The inner loop executes completely whenever outer loop executes.

Example:

```
public class NestedForExample {  
    public static void main(String[] args) {  
        //loop of i  
        for(int i=1;i<=3;i++)  
        {  
            //loop of j  
            for(int j=1;j<=3;j++)  
            {  
                System.out.println(i+" "+j);  
            }  
        }  
    }  
}
```

Output:

```
1 1  
1 2  
1 3  
2 1  
2 2  
2 3  
3 1  
3 2  
3 3
```

Pyramid Example 1:

```
public class PyramidExample {  
    public static void main(String[] args) {  
        for(int i=1;i<=5;i++){  
            for(int j=1;j<=i;j++){  
                System.out.print("* ");  
            }  
            System.out.println();//new line  
        }  
    }  
}
```

Output:

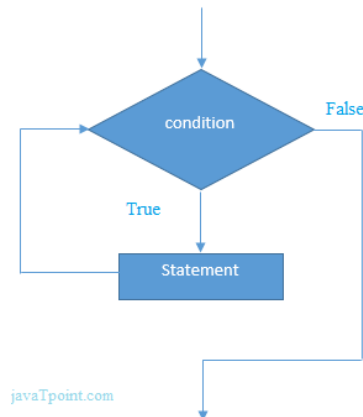
```
*  
  
* *  
  
* * *  
  
* * * *  
  
* * * * *
```

Java While Loop

The Java while loop is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop.

Syntax:

```
while(condition)
{
    //code to be executed
}
```



```
public class WhileExample {
public static void main(String[] args) {
    int i=1;
    while(i<=10){
        System.out.println(i);
        i++;
    }
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

Java do-while

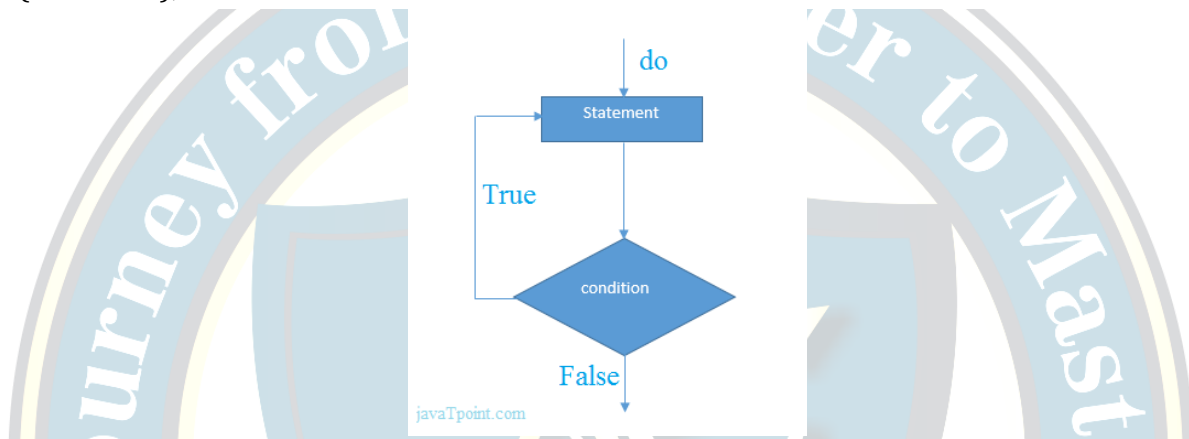
The Java do-while loop is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The Java do-while loop is executed at least once because condition is checked after loop body.

Syntax:

Do

```
{  
//code to be executed  
}while(condition);
```



Example:

```
public class DoWhileExample {  
    public static void main(String[] args) {  
        int i=1;  
        do{  
            System.out.println(i);  
            i++;  
        }while(i<=10);  
    }  
}
```

Output:

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Java Break Statement

When a break statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.

The Java break is used to break loop or switch statement. It breaks the current flow of the program at specified condition.

We can use Java break statement in all types of loops such as for loop, while loop and do-while loop.

Syntax:

jump-statement;

break;

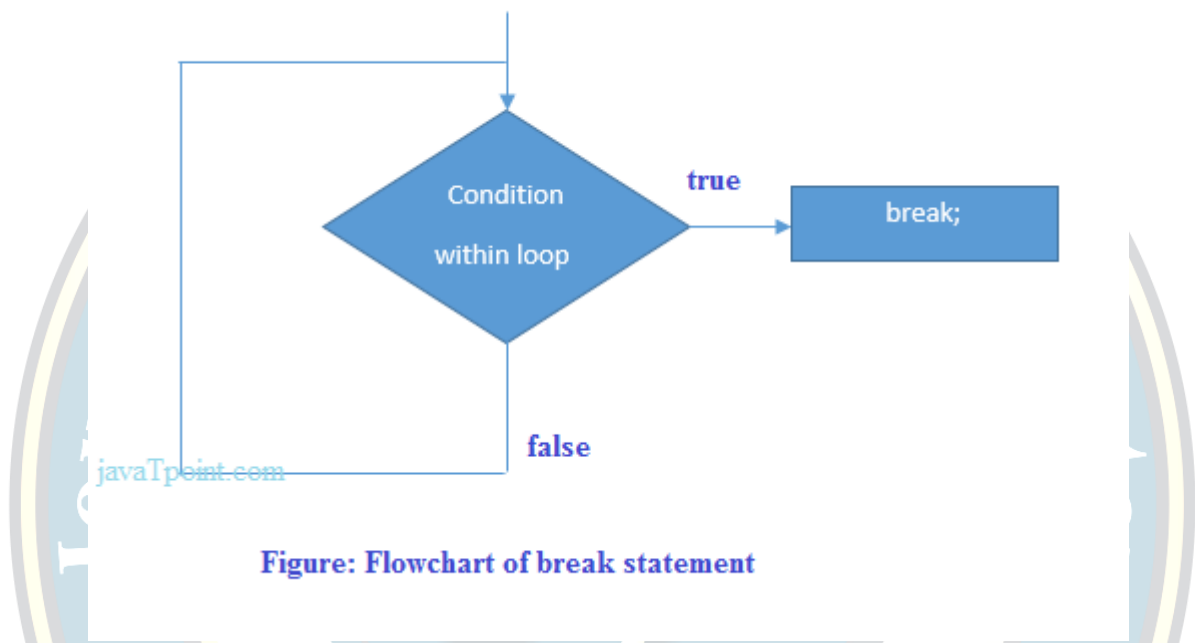


Figure: Flowchart of break statement

Java Break Statement with Loop

Example:

```
public class BreakExample
{
    public static void main(String[] args)
    {
        //using for loop
        for(int i=1;i<=10;i++){
            if(i==5)
            {
                //breaking the loop
                break;
            }
            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
3
4
```

Java Continue Statement

The continue statement is used in loop control structure when you need to jump to the next iteration of the loop immediately. It can be used with for loop or while loop.

The Java continue statement is used to continue the loop. It continues the current flow of the program and skips the remaining code at the specified condition.

We can use Java continue statement in all types of loops such as for loop, while loop and do-while loop.

Syntax:

```
jump-statement;
continue;
```

Example:

```
public class ContinueExample
{
    public static void main(String[] args)
    {
        for(int i=1;i<=10;i++)
        {
            if(i==5)
            {
                continue;//it will skip the rest statement
            }
            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
3
4
6
7
8
9
10
```


As you can see in the above output, 5 is not printed on the console. It is because the loop is continued when it reaches to 5.

0 3 6 9

Java Labeled For Loop

We can have a **name** of each Java **for loop**. To do so, we use **label** before the **for loop**. It is useful if we have **nested for loop** so that we can **break/continue specific for loop**.

Usually, **break** and **continue keywords** breaks/continues the **innermost** for loop **only**.

Syntax:

labelname:

```
for(initialization;condition;incr/decr)
{
    //code to be executed
}
```

Example:

```
public class LabeledForExample {
    public static void main(String[] args)
    {
        aa:
        for(int i=1;i<=3;i++)
        {
            bb:
            for(int j=1;j<=3;j++)
            {
                if(i==2&&j==2)
                {
                    break aa;
                }
                System.out.println(i+" "+j);
            }
        }
    }
}
```

Output:

```
1 1
1 2
1 3
2 1
```

If you use **break bb;**, it will break inner loop only which is the default behavior of any loop.

```
public class LabeledForExample2 {  
    public static void main(String[] args) {  
        aa:  
        for(int i=1;i<=3;i++){  
            bb:  
            for(int j=1;j<=3;j++){  
                if(i==2&& j==2){  
                    break bb;  
                }  
                System.out.println(i+" "+j);  
            }  
        }  
    }  
}
```

Output:

```
1 1  
1 2  
1 3  
2 1  
3 1  
3 2  
3 3
```



Java for-each Loop

In Java, the for-each loop is used to iterate through elements of arrays. It is also known as the enhanced for loop.

Syntax

```
for(dataType item : array) {  
    ...  
}
```

Here,

array - an array

item - each item of array is assigned to this variable

dataType - the data type of the array/collection

Example 1: Print Array Elements

```
class Main {  
    public static void main(String[] args) {  
  
        // create an array  
        int[] numbers = {3, 9, 5, -5};  
  
        // for each loop  
        for (int number: numbers) {  
            System.out.println(number);  
        }  
    }  
}
```

Output

```
3  
9  
5  
-5
```

Here, we have used the for-each loop to print each element of the numbers array one by one.

In the first iteration, item will be 3.

In the second iteration, item will be 9.

In the third iteration, item will be 5.

In the fourth iteration, item will be -5.

Example 2: Sum of Array Elements

// Calculate the sum of all elements of an array

```
class Main {  
    public static void main(String[] args) {  
  
        // an array of numbers  
        int[] numbers = {3, 4, 5, -5, 0, 12};  
        int sum = 0;  
  
        // iterating through each element of the array  
        for (int number: numbers) {  
            sum += number;  
        }  
  
        System.out.println("Sum = " + sum);  
    }  
}
```

Output:

Sum = 19

